

PLANO DE TESTE FUNCIONAL

Banco de Dados - Sistema de Vendas Online

Integrantes

Thiago Nascente	202302630
Vinícius Silva	202302633
Moisés Ferreira	202302624
André Filho	202406913

SGBD: Oracle AI Database 26ai Free

Data-base da execução: 26/09/2026

Sumário

1	Regra de Negócio
2	DER Desenhado
3	Comandos de criação das tabelas
4	Povoamento das tabelas
5	Plano de teste funcional - 40 casos executados

1. Regra de Negócio

O modelo possui oito entidades. Os nomes físicos recebem o prefixo ATS_ para separar a atividade de outros objetos existentes no usuário Oracle THIAGO.

1. ATS_CLIENTE

- `cliente_id` identifica unicamente o cliente.
- `nome` é obrigatório e deve conter de 3 a 100 caracteres úteis.
- `email` é obrigatório, deve possuir formato básico válido e não pode se repetir.
- `telefone`, quando informado, deve conter 10 ou 11 algarismos.
- `data_nascimento` é obrigatória e deve ficar entre 01/01/1900 e 26/09/2008. Esse limite representa 18 anos completos na data-base da execução, 26/09/2026.
- `status_cliente` aceita somente A (ativo) ou I (inativo).

2. ATS_ENDERECO

- Todo endereço pertence a um cliente existente.
- `tipo_endereco` aceita somente RESIDENCIAL ou COMERCIAL.
- `logradouro`, `cidade`, `cep` e `uf` são obrigatórios.
- `numero` deve ser um inteiro entre 1 e 999999.
- `cep` deve conter exatamente 8 algarismos.
- `uf` deve conter exatamente duas letras maiúsculas.
- O par (`endereco_id`, `cliente_id`) é único e permite assegurar que o endereço usado no pedido pertence ao cliente do pedido.

3. ATS_CATEGORIA

- `categoria_id` identifica unicamente a categoria.
- `nome` é obrigatório, deve conter de 3 a 80 caracteres e não pode se repetir.
- `ativo` aceita somente S ou N.
- Uma categoria com produtos associados não pode ser excluída.

4. ATS_PRODUTO

- Todo produto pertence a uma categoria existente.
- `sku` é obrigatório e não pode se repetir.
- `nome` é obrigatório e deve conter de 3 a 120 caracteres.
- `preco` deve ficar entre R\$ 0,01 e R\$ 999.999,99.
- `estoque` deve ser inteiro e ficar entre 0 e 99.999.999 unidades.
- `ativo` aceita somente S ou N.

5. ATS_PEDIDO

- Todo pedido pertence a um cliente existente e usa um endereço existente do mesmo cliente.
- `data_pedido` é obrigatória e recebe a data atual quando não informada.
- `status_pedido` aceita ABERTO, PAGO, ENVIADO, ENTREGUE ou CANCELADO.
- `valor_total` deve ficar entre zero e R\$ 9.999.999.999,99.
- Um cliente com pedidos associados não pode ser excluído.

6. ATS_ITEM_PEDIDO

- O item é identificado pelo par (`pedido_id`, `produto_id`).
- Todo item referencia um pedido e um produto existentes.
- `quantidade` deve ser inteira e ficar entre 1 e 999.
- `preco_unitario` deve ficar entre R\$ 0,01 e R\$ 999.999,99.
- `desconto_percentual` deve ficar entre 0% e 100%.

7. ATS_PAGAMENTO

- Todo pagamento pertence a um pedido existente.
- Cada pedido pode ter, no máximo, um pagamento.
- `forma_pagamento` aceita PIX, CARTAO, BOLETO ou DINHEIRO.
- `valor` deve ser positivo.
- `parcelas` deve ser um inteiro entre 1 e 12.
- `status_pagamento` aceita PENDENTE, APROVADO, RECUSADO ou ESTORNADO.
- O valor do pagamento deve ser exatamente igual ao `valor_total` do pedido. A chave estrangeira composta (`pedido_id`, `valor`) aplica a regra sem depender de gatilhos.

8. ATS_ENTREGA

- Toda entrega pertence a um pedido existente.
- Cada pedido pode ter, no máximo, uma entrega.
- `transportadora` é obrigatória.
- `codigo_rastreio`, quando informado, não pode se repetir.
- `status_entrega` aceita PREPARANDO, ENVIADO, ENTREGUE ou DEVOLVIDO.
- A data prevista não pode ser anterior à data de envio.
- A data efetiva de entrega não pode ser anterior à data de envio.
- Uma entrega com status ENTREGUE deve possuir `data_entrega`.

2. DER Desenhado

O diagrama abaixo apresenta as oito entidades, seus principais atributos e os relacionamentos implementados no Oracle.

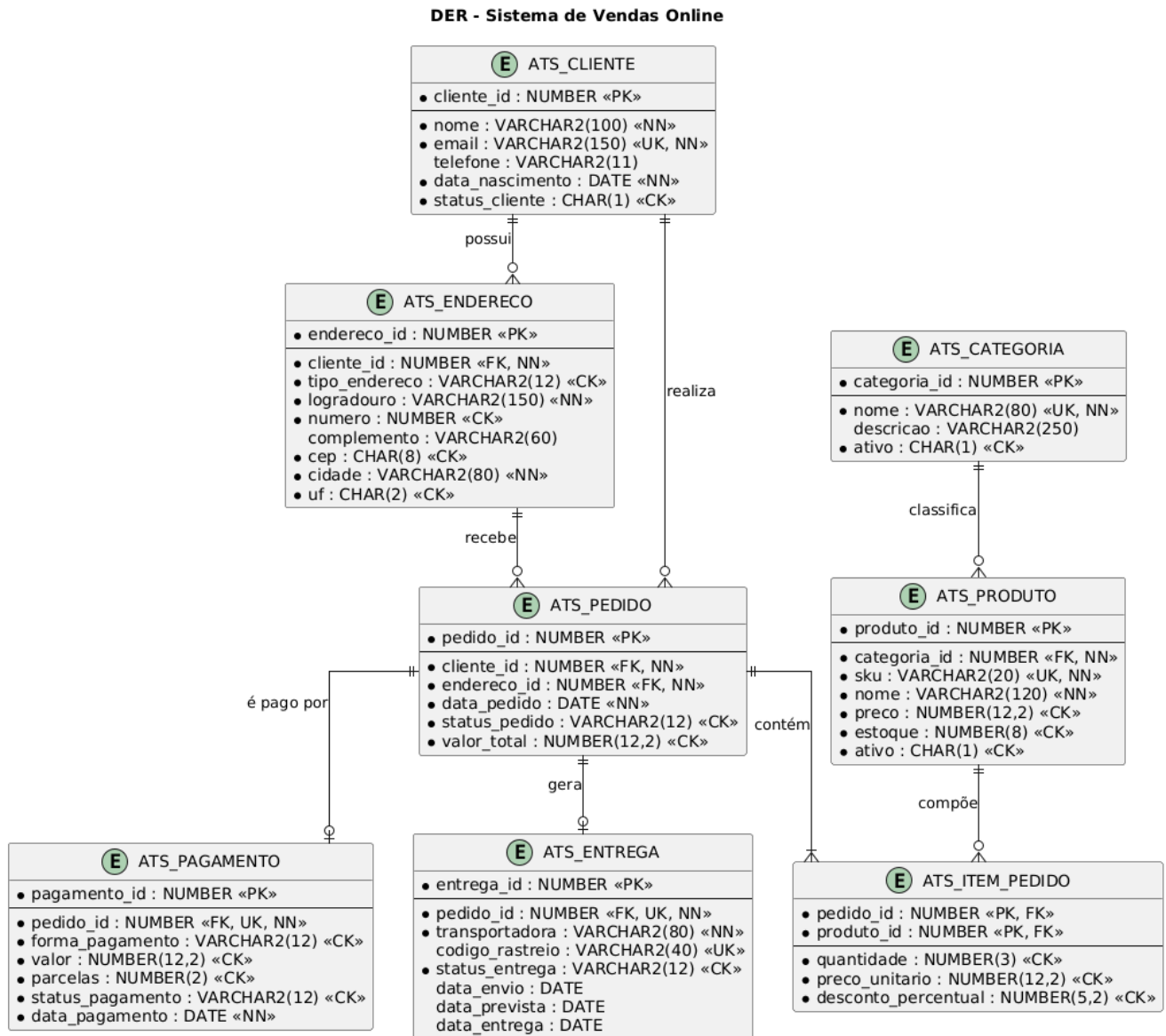


Figura 1 - DER do Sistema de Vendas Online.

3. Comandos de criação das tabelas

Ambiente e execução

- SGBD: Oracle AI Database 26ai Free
- Contêiner: FREEPDB1
- Usuário: THIAGO
- Data da execução: 26/09/2026
- Resultado: oito tabelas criadas com sucesso.

O arquivo executável completo é `scripts/01\_criar\_tabelas.sql`. No SQL*Plus, a criação foi iniciada com:

```
@scripts/01_criar_tabelas.sql
```

Comandos DDL executados

```
CREATE TABLE ats_cliente (  
  cliente_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  nome VARCHAR2(100) NOT NULL,  
  email VARCHAR2(150) NOT NULL,  
  telefone VARCHAR2(11),  
  data_nascimento DATE NOT NULL,  
  status_cliente CHAR(1) DEFAULT 'A' NOT NULL,  
  CONSTRAINT pk_ats_cliente PRIMARY KEY (cliente_id),  
  CONSTRAINT uk_ats_cliente_email UNIQUE (email),  
  CONSTRAINT ck_ats_cliente_nome CHECK (LENGTH(TRIM(nome)) BETWEEN 3 AND 100),  
  CONSTRAINT ck_ats_cliente_email CHECK (REGEXP_LIKE(email, '^[^@ ]+@[^@ ]+[.][^@ ]+$')),  
  CONSTRAINT ck_ats_cliente_tel CHECK (telefone IS NULL OR REGEXP_LIKE(telefone, '[0-9]{10,11}$')),  
  CONSTRAINT ck_ats_cliente_nasc CHECK (data_nascimento BETWEEN DATE '1900-01-01' AND DATE '2008-09-26'),  
  CONSTRAINT ck_ats_cliente_status CHECK (status_cliente IN ('A', 'I'))  
);  
  
CREATE TABLE ats_endereco (  
  endereco_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  cliente_id NUMBER NOT NULL,  
  tipo_endereco VARCHAR2(12) NOT NULL,  
  logradouro VARCHAR2(150) NOT NULL,  
  numero NUMBER(6) NOT NULL,  
  complemento VARCHAR2(60),  
  cep CHAR(8) NOT NULL,  
  cidade VARCHAR2(80) NOT NULL,  
  uf CHAR(2) NOT NULL,  
  CONSTRAINT pk_ats_endereco PRIMARY KEY (endereco_id),  
  CONSTRAINT uk_ats_endereco_cli UNIQUE (endereco_id, cliente_id),  
  CONSTRAINT fk_ats_endereco_cliente FOREIGN KEY (cliente_id) REFERENCES ats_cliente (cliente_id),  
  CONSTRAINT ck_ats_endereco_tipo CHECK (tipo_endereco IN ('RESIDENCIAL', 'COMERCIAL')),  
  CONSTRAINT ck_ats_endereco_logr CHECK (LENGTH(TRIM(logradouro)) BETWEEN 3 AND 150),  
  CONSTRAINT ck_ats_endereco_num CHECK (numero BETWEEN 1 AND 999999 AND numero = TRUNC(numero)),  
  CONSTRAINT ck_ats_endereco_cep CHECK (REGEXP_LIKE(cep, '[0-9]{8}$')),  
  CONSTRAINT ck_ats_endereco_uf CHECK (REGEXP_LIKE(uf, '[A-Z]{2}$'))  
);  
  
CREATE TABLE ats_categoria (  
  categoria_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  nome VARCHAR2(80) NOT NULL,  
  descricao VARCHAR2(250),  
  ativo CHAR(1) DEFAULT 'S' NOT NULL,  
  CONSTRAINT pk_ats_categoria PRIMARY KEY (categoria_id),  
  CONSTRAINT uk_ats_categoria_nome UNIQUE (nome),  
  CONSTRAINT ck_ats_categoria_nome CHECK (LENGTH(TRIM(nome)) BETWEEN 3 AND 80),  
  CONSTRAINT ck_ats_categoria_ativo CHECK (ativo IN ('S', 'N'))  
);  
  
CREATE TABLE ats_produto (  
  produto_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  categoria_id NUMBER NOT NULL,  
  sku VARCHAR2(20) NOT NULL,  
  nome VARCHAR2(120) NOT NULL,  
  preco NUMBER(12,2) NOT NULL,  
  estoque NUMBER(8) DEFAULT 0 NOT NULL,  
  ativo CHAR(1) DEFAULT 'S' NOT NULL,  
  CONSTRAINT pk_ats_produto PRIMARY KEY (produto_id),  
  CONSTRAINT uk_ats_produto_sku UNIQUE (sku),  
  CONSTRAINT fk_ats_produto_categoria FOREIGN KEY (categoria_id) REFERENCES ats_categoria (categoria_id),  
  CONSTRAINT ck_ats_produto_nome CHECK (LENGTH(TRIM(nome)) BETWEEN 3 AND 120),  
  CONSTRAINT ck_ats_produto_preco CHECK (preco BETWEEN 0.01 AND 999999.99),  
  CONSTRAINT ck_ats_produto_estoque CHECK (estoque BETWEEN 0 AND 99999999 AND estoque = TRUNC(estoque)),
```

```

CONSTRAINT ck_ats_produto_ativo CHECK (ativo IN ('S', 'N'))
);

CREATE TABLE ats_pedido (
    pedido_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
    cliente_id NUMBER NOT NULL,
    endereco_id NUMBER NOT NULL,
    data_pedido DATE DEFAULT SYSDATE NOT NULL,
    status_pedido VARCHAR2(12) DEFAULT 'ABERTO' NOT NULL,
    valor_total NUMBER(12,2) NOT NULL,
    CONSTRAINT pk_ats_pedido PRIMARY KEY (pedido_id),
    CONSTRAINT uk_ats_pedido_id_total UNIQUE (pedido_id, valor_total),
    CONSTRAINT fk_ats_pedido_cliente FOREIGN KEY (cliente_id) REFERENCES ats_cliente (cliente_id),
    CONSTRAINT fk_ats_pedido_end_cli FOREIGN KEY (endereco_id, cliente_id)
        REFERENCES ats_endereco (endereco_id, cliente_id),
    CONSTRAINT ck_ats_pedido_status CHECK (status_pedido IN ('ABERTO', 'PAGO', 'ENVIADO', 'ENTREGUE',
        'CANCELADO')),
    CONSTRAINT ck_ats_pedido_total CHECK (valor_total BETWEEN 0 AND 999999999.99)
);

CREATE TABLE ats_item_pedido (
    pedido_id NUMBER NOT NULL,
    produto_id NUMBER NOT NULL,
    quantidade NUMBER(3) NOT NULL,
    preco_unitario NUMBER(12,2) NOT NULL,
    desconto_percentual NUMBER(5,2) DEFAULT 0 NOT NULL,
    CONSTRAINT pk_ats_item_pedido PRIMARY KEY (pedido_id, produto_id),
    CONSTRAINT fk_ats_item_pedido FOREIGN KEY (pedido_id) REFERENCES ats_pedido (pedido_id),
    CONSTRAINT fk_ats_item_produto FOREIGN KEY (produto_id) REFERENCES ats_produto (produto_id),
    CONSTRAINT ck_ats_item_qtd CHECK (quantidade BETWEEN 1 AND 999 AND quantidade = TRUNC(quantidade)),
    CONSTRAINT ck_ats_item_preco CHECK (preco_unitario BETWEEN 0.01 AND 999999.99),
    CONSTRAINT ck_ats_item_desconto CHECK (desconto_percentual BETWEEN 0 AND 100)
);

CREATE TABLE ats_pagamento (
    pagamento_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
    pedido_id NUMBER NOT NULL,
    forma_pagamento VARCHAR2(12) NOT NULL,
    valor NUMBER(12,2) NOT NULL,
    parcelas NUMBER(2) DEFAULT 1 NOT NULL,
    status_pagamento VARCHAR2(12) DEFAULT 'PENDENTE' NOT NULL,
    data_pagamento DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_ats_pagamento PRIMARY KEY (pagamento_id),
    CONSTRAINT uk_ats_pagamento_pedido UNIQUE (pedido_id),
    CONSTRAINT fk_ats_pagamento_pedido FOREIGN KEY (pedido_id) REFERENCES ats_pedido (pedido_id),
    CONSTRAINT fk_ats_pagamento_total FOREIGN KEY (pedido_id, valor)
        REFERENCES ats_pedido (pedido_id, valor_total),
    CONSTRAINT ck_ats_pagamento_forma CHECK (forma_pagamento IN ('PIX', 'CARTAO', 'BOLETO', 'DINHEIRO')),
    CONSTRAINT ck_ats_pagamento_valor CHECK (valor BETWEEN 0.01 AND 999999999.99),
    CONSTRAINT ck_ats_pagamento_parcelas CHECK (parcelas BETWEEN 1 AND 12 AND parcelas = TRUNC(parcelas)),
    CONSTRAINT ck_ats_pagamento_status CHECK (status_pagamento IN ('PENDENTE', 'APROVADO', 'RECUSADO',
        'ESTORNADO'))
);

CREATE TABLE ats_entrega (
    entrega_id NUMBER GENERATED BY DEFAULT AS IDENTITY,
    pedido_id NUMBER NOT NULL,
    transportadora VARCHAR2(80) NOT NULL,
    codigo_rastreio VARCHAR2(40),
    status_entrega VARCHAR2(12) DEFAULT 'PREPARANDO' NOT NULL,
    data_envio DATE,
    data_prevista DATE,
    data_entrega DATE,
    CONSTRAINT pk_ats_entrega PRIMARY KEY (entrega_id),
    CONSTRAINT uk_ats_entrega_pedido UNIQUE (pedido_id),
    CONSTRAINT uk_ats_entrega_rastreio UNIQUE (codigo_rastreio),
    CONSTRAINT fk_ats_entrega_pedido FOREIGN KEY (pedido_id) REFERENCES ats_pedido (pedido_id),
    CONSTRAINT ck_ats_entrega_status CHECK (status_entrega IN ('PREPARANDO', 'ENVIADO', 'ENTREGUE', 'DEVOLVIDO')),
    CONSTRAINT ck_ats_entrega_prev CHECK (data_prevista IS NULL OR data_envio IS NULL OR data_prevista >=
        data_envio),
    CONSTRAINT ck_ats_entrega_data CHECK (data_entrega IS NULL OR (data_envio IS NOT NULL AND data_entrega >=
        data_envio)),
    CONSTRAINT ck_ats_entrega_concl CHECK (status_entrega <> 'ENTREGUE' OR data_entrega IS NOT NULL)
);

```

Evidência

Os scripts SQL utilizados na criação, no povoamento e na execução dos testes estão disponíveis no repositório:

<https://github.com/ThiagoNascente/DER-TESTE-DE-SOFTWARE-ATV2/tree/main/scripts>

4. Povoamento das tabelas

Execução

O povoamento foi executado no SQL*Plus e confirmado com COMMIT pelo comando:

```
@scripts/02_popular_banco.sql
```

O script executável completo está em `scripts/02\_popular\_banco.sql`, e a saída integral está em `scripts/execucao-banco.log`.

Registros inseridos

Tabela	Quantidade	Identificadores e conteúdo principal
ATS_CLIENTE	4	IDs 1 a 4; Ana, Bruno, Carla e Diego
ATS_ENDERECO	5	IDs 101 a 105; endereços em SP, RJ, MG e PR
ATS_CATEGORIA	4	IDs 201 a 204; Eletrônicos, Casa e Cozinha, Livros e Esportes
ATS_PRODUTO	6	IDs 301 a 306; preços de R\$ 45,00 a R\$ 3.999,90
ATS_PEDIDO	4	IDs 401 a 404; totais de R\$ 225,00 a R\$ 4.161,72
ATS_ITEM_PEDIDO	5	Itens associados aos quatro pedidos
ATS_PAGAMENTO	2	IDs 501 e 502; valores iguais aos totais dos pedidos 401 e 402
ATS_ENTREGA	2	IDs 601 e 602; uma entrega enviada e uma em preparação

Conferência executada

```
SELECT 'ATS_CLIENTE' tabela, COUNT(*) quantidade FROM ats_cliente UNION ALL
SELECT 'ATS_ENDERECO', COUNT(*) FROM ats_endereco UNION ALL
SELECT 'ATS_CATEGORIA', COUNT(*) FROM ats_categoria UNION ALL
SELECT 'ATS_PRODUTO', COUNT(*) FROM ats_produto UNION ALL
SELECT 'ATS_PEDIDO', COUNT(*) FROM ats_pedido UNION ALL
SELECT 'ATS_ITEM_PEDIDO', COUNT(*) FROM ats_item_pedido UNION ALL
SELECT 'ATS_PAGAMENTO', COUNT(*) FROM ats_pagamento UNION ALL
SELECT 'ATS_ENTREGA', COUNT(*) FROM ats_entrega;
```

Resultado obtido: 4, 5, 4, 6, 4, 5, 2, 2, respectivamente. Os 40 testes usam SAVEPOINT e ROLLBACK TO SAVEPOINT; por isso, não alteram esse povoamento.

5. Plano de teste funcional

A execução consolidada contém 40 casos de teste distribuídos entre integridade referencial, integridade de atributos, regras semânticas, unicidade e valores-limite.

Casos executados	Resultados OK	Resultados ERRO
40	40	0

A página seguinte reproduz integralmente o arquivo Plano-de-teste-atividade-2.pdf, com entradas, resultados esperados, resultados obtidos, status e observações.

Plano de teste funcional do banco de dados

Sistema	Vendas Online		SGBD	Oracle AI Database 26ai Free		Data-base	26/09/2026
Total de CTs	40	OK	40	ERRO	0	Resultado	APROVADO
Evidência	scripts/resultados-testes.txt — saída real do SQL*Plus						

Tabela	CT	Restrição / regra	Dado de entrada / script	Resultado esperado	Resultado obtido	Status	Observação
Critério: integridade referencial							
ATS_ENDERECO	1	O cliente referenciado deve existir.	Inserir endereco com cliente_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: cliente inexistente bloqueado pela chave estrangeira.	OK	Classe inválida: chave-pai inexistente.
ATS_PRODUTO	2	A categoria referenciada deve existir.	Inserir produto com categoria_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: categoria inexistente bloqueada pela chave estrangeira.	OK	Classe inválida: categoria inexistente.
ATS_PEDIDO	3	O endereço deve pertencer ao mesmo cliente do pedido.	cliente_id = 1 e endereco_id = 102, pertencente ao cliente 2. scripts/03_testes_integridade_referencial.sql	Rejeitar o par endereço/cliente incompatível.	ORA-02291: par endereco/cliente incompatível bloqueado.	OK	Valida a chave estrangeira composta.
ATS_CATEGORIA	4	Categoria com produtos associados não pode ser excluída.	Excluir categoria_id = 201, referenciada por produtos. scripts/03_testes_integridade_referencial.sql	Rejeitar a exclusão por dependência.	ORA-02292: categoria referenciada por produtos não foi excluída.	OK	Classe inválida: exclusão de chave-pai em uso.
ATS_ITEM_PEDIDO	5	O pedido referenciado deve existir.	Inserir item com pedido_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: pedido inexistente bloqueado pela chave estrangeira.	OK	Classe inválida: pedido inexistente.
ATS_ITEM_PEDIDO	6	O produto referenciado deve existir.	Inserir item com produto_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: produto inexistente bloqueado pela chave estrangeira.	OK	Classe inválida: produto inexistente.
ATS_PAGAMENTO	7	O pedido do pagamento deve existir.	Inserir pagamento com pedido_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: pagamento sem pedido foi bloqueado.	OK	Classe inválida: pagamento órfão.
ATS_ENTREGA	8	O pedido da entrega deve existir.	Inserir entrega com pedido_id = 99999. scripts/03_testes_integridade_referencial.sql	Rejeitar com violação de chave estrangeira.	ORA-02291: entrega sem pedido foi bloqueada.	OK	Classe inválida: entrega órfã.
Critério: integridade de atributos — classes de equivalência e valores-limite							
ATS_CLIENTE	9	nome não pode ser nulo.	nome = NULL. scripts/04_testes_atributos_limites.sql	Rejeitar o registro por campo obrigatório.	ORA-01400: nome nulo bloqueado.	OK	Classe inválida: ausência de valor obrigatório.
ATS_CLIENTE	10	data_nascimento >= 01/01/1900.	data_nascimento = 01/01/1900. scripts/04_testes_atributos_limites.sql	Aceitar o registro no limite inferior.	Data 1900-01-01 aceita no limite inferior válido.	OK	Valor de fronteira inferior válido.
ATS_CLIENTE	11	data_nascimento >= 01/01/1900.	data_nascimento = 31/12/1899. scripts/04_testes_atributos_limites.sql	Rejeitar o registro abaixo do limite.	ORA-02290: data 1899-12-31 bloqueada abaixo do limite.	OK	Um dia abaixo da fronteira inferior.
ATS_CLIENTE	12	Cliente deve ter 18 anos na data-base 26/09/2026.	data_nascimento = 27/09/2008. scripts/04_testes_atributos_limites.sql	Rejeitar o registro um dia abaixo de 18 anos.	ORA-02290: nascimento em 2008-09-27, um dia abaixo do limite de 18 anos, foi bloqueado.	OK	Um dia acima da data máxima permitida.
ATS_ENDERECO	13	cep deve conter exatamente 8 algarismos.	cep = '1234567'. scripts/04_testes_atributos_limites.sql	Rejeitar CEP com 7 algarismos.	ORA-02290: CEP com 7 dígitos foi bloqueado.	OK	Classe inválida: comprimento menor que 8.
ATS_ENDERECO	14	numero deve ser inteiro entre 1 e 999999.	numero = 1. scripts/04_testes_atributos_limites.sql	Aceitar o limite inferior.	Numero 1 aceito no limite inferior válido.	OK	Valor de fronteira inferior válido.
ATS_PRODUTO	15	preco deve ser >= 0,01.	preco = 0. scripts/04_testes_atributos_limites.sql	Rejeitar preço zero.	ORA-02290: preco zero bloqueado.	OK	Um centavo abaixo da fronteira inferior.
ATS_PRODUTO	16	preco >= 0,01 e estoque >= 0.	preco = 0,01 e estoque = 0. scripts/04_testes_atributos_limites.sql	Aceitar ambos os limites inferiores.	Preco 0,01 e estoque zero aceitos nos limites válidos.	OK	Duas fronteiras válidas no mesmo registro.
ATS_PRODUTO	17	estoque deve ser >= 0.	estoque = -1. scripts/04_testes_atributos_limites.sql	Rejeitar estoque negativo.	ORA-02290: estoque -1 bloqueado abaixo do limite.	OK	Uma unidade abaixo da fronteira inferior.
ATS_ITEM_PEDIDO	18	quantidade deve ficar entre 1 e 999.	quantidade = 0. scripts/04_testes_atributos_limites.sql	Rejeitar quantidade zero.	ORA-02290: quantidade zero bloqueada.	OK	Uma unidade abaixo da fronteira inferior.
ATS_ITEM_PEDIDO	19	quantidade deve ficar entre 1 e 999.	quantidade = 1. scripts/04_testes_atributos_limites.sql	Aceitar o limite inferior.	Quantidade 1 aceita no limite inferior válido.	OK	Valor de fronteira inferior válido.
ATS_ITEM_PEDIDO	20	desconto_percentual deve ficar entre 0 e 100.	desconto_percentual = 100. scripts/04_testes_atributos_limites.sql	Aceitar o limite superior.	Desconto 100 por cento aceito no limite superior válido.	OK	Valor de fronteira superior válido.
ATS_ITEM_PEDIDO	21	desconto_percentual deve ficar entre 0 e 100.	desconto_percentual = 100,01. scripts/04_testes_atributos_limites.sql	Rejeitar valor acima de 100.	ORA-02290: desconto 100,01 bloqueado acima do limite.	OK	0,01 acima da fronteira superior.
ATS_PAGAMENTO	22	parcelas deve ser inteiro entre 1 e 12.	parcelas = 13. scripts/04_testes_atributos_limites.sql	Rejeitar valor acima de 12.	ORA-02290: 13 parcelas bloqueadas acima do limite.	OK	Uma parcela acima da fronteira superior.
ATS_ENTREGA	23	Status ENTREGUE exige data_entrega.	status_entrega = 'ENTREGUE'; data_entrega = NULL. scripts/04_testes_atributos_limites.sql	Rejeitar a combinação inconsistente.	ORA-02290: status ENTREGUE sem data efetiva foi bloqueado.	OK	Classe inválida: estado concluído sem data.
ATS_CATEGORIA	24	ativo aceita somente S ou N.	ativo = 'X'. scripts/04_testes_atributos_limites.sql	Rejeitar valor fora do domínio.	ORA-02290: valor X fora do domínio foi bloqueado.	OK	Classe inválida de domínio enumerado.
Critério: integridade semântica, unicidade e dependência de dados							
ATS_PAGAMENTO	25	valor do pagamento deve igualar o total do pedido.	pedido 403: valor_total = 389,70; pagamento = 300,00. scripts/05_testes_semanticos.sql	Rejeitar o pagamento divergente.	ORA-02291: pagamento divergente do total do pedido foi bloqueado.	OK	Dependência entre valor e pedido.
ATS_PAGAMENTO	26	valor do pagamento deve igualar o total do pedido.	pedido 403: pagamento = 389,70. scripts/05_testes_semanticos.sql	Aceitar o pagamento de mesmo valor.	Pagamento aprovado igual ao total do pedido foi aceito.	OK	Classe válida da dependência entre dados.
ATS_ENTREGA	27	data_prevista >= data_envio.	envio = 25/09/2026; prevista = 24/09/2026. scripts/05_testes_semanticos.sql	Rejeitar data prevista anterior ao envio.	ORA-02290: data prevista anterior ao envio foi bloqueada.	OK	Ordem cronológica inválida.
ATS_ENTREGA	28	data_entrega >= data_envio.	envio = 25/09/2026; entrega = 24/09/2026. scripts/05_testes_semanticos.sql	Rejeitar entrega anterior ao envio.	ORA-02290: data efetiva anterior ao envio foi bloqueada.	OK	Ordem cronológica inválida.
ATS_CLIENTE	29	email deve ser único.	email = 'ana.souza@example.com', já cadastrado. scripts/05_testes_semanticos.sql	Rejeitar o email duplicado.	ORA-00001: email duplicado foi bloqueado.	OK	Classe inválida de unicidade.
ATS_PAGAMENTO	30	Cada pedido pode ter no máximo um pagamento.	Inserir segundo pagamento para pedido_id = 401. scripts/05_testes_semanticos.sql	Rejeitar o segundo pagamento.	ORA-00001: segundo pagamento do pedido foi bloqueado.	OK	Dependência 1.0..1 protegida por unicidade.
ATS_CLIENTE	31	Cliente com endereços ou pedidos não pode ser excluído.	Excluir cliente_id = 1. scripts/05_testes_semanticos.sql	Rejeitar a exclusão por dependências.	ORA-02292: cliente com endereços e pedidos não foi excluído.	OK	Proteção contra registros órfãos.
ATS_PEDIDO	32	valor_total deve ser >= 0.	valor_total = -0,01. scripts/05_testes_semanticos.sql	Rejeitar total negativo.	ORA-02290: total negativo do pedido foi bloqueado.	OK	0,01 abaixo da fronteira inferior.
Critério: valores-limite adicionais							
ATS_CLIENTE	33	nome deve conter entre 3 e 100 caracteres.	nome = 'AB', com 2 caracteres. scripts/06_testes_limites_adicionais.sql	Rejeitar o nome abaixo do comprimento mínimo.	ORA-02290: nome com 2 caracteres foi bloqueado abaixo do limite.	OK	Um caractere abaixo da fronteira inferior.
ATS_CLIENTE	34	data_nascimento <= 26/09/2008.	data_nascimento = 26/09/2008. scripts/06_testes_limites_adicionais.sql	Aceitar a data no limite superior.	Nascimento em 2008-09-26 aceito no limite superior válido.	OK	Valor de fronteira superior válido.
ATS_ENDERECO	35	numero deve ser inteiro entre 1 e 999999.	numero = 999999. scripts/06_testes_limites_adicionais.sql	Aceitar o limite superior.	Numero 999999 aceito no limite superior válido.	OK	Valor de fronteira superior válido.
ATS_ENDERECO	36	numero deve ser inteiro entre 1 e 999999.	numero = 1000000. scripts/06_testes_limites_adicionais.sql	Rejeitar o valor acima de 999999.	Valor 1000000 bloqueado acima do limite do numero do endereco.	OK	Uma unidade acima da fronteira superior.
ATS_PRODUTO	37	preco deve ficar entre 0,01 e 999999,99.	preco = 999999,99. scripts/06_testes_limites_adicionais.sql	Aceitar o limite superior.	Preco 999999,99 aceito no limite superior válido.	OK	Valor de fronteira superior válido.
ATS_PRODUTO	38	preco deve ficar entre 0,01 e 999999,99.	preco = 1000000,00. scripts/06_testes_limites_adicionais.sql	Rejeitar um centavo acima do máximo.	ORA-02290: preco 1000000,00 bloqueado acima do limite.	OK	0,01 acima da fronteira superior.
ATS_ITEM_PEDIDO	39	quantidade deve ficar entre 1 e 999.	quantidade = 999. scripts/06_testes_limites_adicionais.sql	Aceitar o limite superior.	Quantidade 999 aceita no limite superior válido.	OK	Valor de fronteira superior válido.
ATS_ITEM_PEDIDO	40	quantidade deve ficar entre 1 e 999.	quantidade = 1000. scripts/06_testes_limites_adicionais.sql	Rejeitar uma unidade acima do máximo.	Quantidade 1000 bloqueada acima do limite do item.	OK	Uma unidade acima da fronteira superior.